

Fund4Life: A Community-Driven Emergency Medical Support System

An Application Development - I Report Submitted
In partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

**U Charan - 23N31A66W5
T Krishna - 23N31A66V7
V Rithika - 23N31A66X4**

Under the Guidance of
T Jayasri
Assistant Professor



**DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING)**
MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY
(Autonomous Institution-UGC, Govt. of India)
(Affiliated to JNTU, Hyderabad), ACCREDITED by AICTE-NBA
Maisammaguda, Dhulapally post, Secunderabad-500014.
2025-2026

DECLARATION

I hereby declare that the project entitled “**Fund4Life: A Community-Driven Emergency Medical Support System**” submitted to **Malla Reddy College of Engineering and Technology**,affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of **Bachelor of Technology in Computer Science and Engineering- Artificial Intelligence and Machine Learning** is a result of original research work done by us.

It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

U Charan(21N31A66W5)

T Krishna(23N31A66V7)

V Rithika(23N31A66X4)



MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS INSTITUTION - UGC, GOVT. OF INDIA)

Affiliated to JNTUH; Approved by AICTE, NBA-Tier 1 & NAAC with A-GRADE | ISO 9001:2015

CERTIFICATE

This is to certify that this is the Bonafide record of the project titled “**Fund4Life: A Community-Driven Emergency Medical Support System**” submitted by **U charan** (23N31A66W5), **T krishna** (23N31A66Y7), **V Rithika**(23N31A66X4) of B.Tech in the partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering- Artificial Intelligence and Machine Learning**, Dept. of CSE(AI&ML) during the year 2025-2026. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

T Jayasri

Assistant Professor

INTERNAL GUIDE

Dr. A. Nagaraju

Professor (AIML&ET)

HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

Date of Viva-Voce Examination held on: _____

ACKNOWLEDGEMENT

The Project Application Development - I work carried out by our team in the Department of CSE (Artificial Intelligence and Machine Learning), Malla Reddy College of Engineering and Technology, Hyderabad. **This work is original and has not been submitted in part or full for any degree or diploma of any other university.**

We wish to acknowledge our sincere thanks to our project guide **T Jayasri**, Assistant Professor for formulation of the problem, analysis, guidance and his continuous supervision during the course of work.

We acknowledge our sincere thanks to **Dr. Vaka Murali Mohan**, Principal, **Dr. A NAGARAJU** Head of the Department and **T. Hari Babu**, Coordinator, faculty members of CSE(AI&ML) Department for their kind cooperation in making this Application Development - I work a success.

We extend our gratitude to **Sri. Ch. Malla Reddy**, Founder Chairman MRGI and **Sri. Ch. Mahender Reddy**, Secretary MRGI, **Dr. Ch. Bhadra Reddy**, President MRGI, **Sri. Ch. Shalini Reddy**, Director MRGI, **Sri. P. Praveen Reddy**, Director MRGI, for their kind cooperation in providing the infrastructure for completion of Application Development - I

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the CSE (Artificial Intelligence and Machine Learning) Department for their support in making this project work a success.

By:

U Charan- 21N31A66W5.....

T Krishna-23N31A66Y7.....

V Rithika-23N31A66X4.....

ABSTRACT

The rising cost of healthcare has created severe financial challenges, especially for underprivileged patients who often struggle to access timely treatment due to lack of immediate funds. On the other hand, many donors who genuinely wish to help face issues of trust and transparency. To address these challenges, Fund4Life is designed as a community-driven platform that bridges the gap between donors, NGOs, and patients. Donors can contribute regularly starting from as little as ₹100 per month, creating a sustainable pool of funds, while patients in emergencies can register, submit medical documents, and request financial aid. An admin verification system, supported by NGOs and volunteers, ensures the authenticity of cases and builds a transparent ecosystem.

Fund4Life focuses on creating a trusted and inclusive network where donors and recipients belong to the same community. The platform includes donor registration, NGO approval management, emergency request submission, fund allocation, and transparent monitoring of transactions. Future enhancements include AI-based document analysis to speed up case verification and ensure fair distribution of resources. By encouraging recurring donations, volunteer participation, and NGO collaboration, Fund4Life goes beyond being a donation service it becomes a lifeline network providing timely medical assistance, financial support, and hope, ensuring that no one is denied treatment due to lack of funds.

TABLE OF CONTENTS

CONTENTS	Page No.
1. INTRODUCTION	1
1.1. Problem Statements	1
1.2. Objectives	1
1.3. Summary	1
2. LITERATURE SURVEY	3
2.1. Existing System	3
2.2. Proposed System	4
3. SYSTEM REQUIREMENTS	5
3.1. Introduction	5
3.2. Software and Hardware Requirement	6
3.3. Functional and Non-Functional Requirements	6
3.4. Other Requirements	8
3.5. Summary	8
4. SYSTEM DESIGN	9
4.1. Introduction	9
4.2. Architecture Diagram	10
4.3. UML Diagrams / DFD	11
4.4. Summary	14
5. IMPLEMENTATION	15
5.1. Algorithms	15
5.2. Architectural Components	15
5.3. Feature Extraction	16
5.4. Packages/Libraries Used	17
5.5. Code	19
5.6. Output Screens	41
6. SYSTEM TESTING	50
6.1. Introduction	50
6.2. Test Cases	50
6.3. Results and Discussions	51
6.4. Performance Evaluation	51
6.5. Summary	52
7. CONCLUSION & FUTURE ENHANCEMENTS	53
8. REFERENCES	55

LIST OF FIGURES

Fig. No	Figure Title	Page no.
4.1	Architecture Diagram	10
4.2.1	Use Case Diagram	11
4.2.2	Class Diagram	12
4.2.3	Sequential Diagram	13
4.2.4	Activity Diagram	14

LIST OF TABLES

S. No	Table Name	Page No
6.2	Sample test cases	50

LIST OF ABBREVIATIONS

S. No	ABBREVIATIONS
1.	NGO –Non-Governmental Organization
2.	API – Application Programming Interface
3.	MUI -Material-UI
4.	SQL – Structured Query Language

CHAPTER 1

INTRODUCTION

The Fund4Life project is a community-driven digital platform designed to bridge the gap between donors and emergency recipients. It enables individuals to contribute small monthly donations starting from ₹100, which are collectively used to support verified medical emergency cases and NGOs.

The platform ensures trust, transparency, and inclusivity, allowing both donors and recipients to coexist in a single verified ecosystem. Built with a Flask backend and React-based Admin Dashboard, Fund4Life provides real-time management, document verification, and fund allocation to genuine cases, fostering a reliable donation environment.

1.1 Problem Statements:

- Traditional donation systems often lack transparency and trust between donors and recipients.
- Verification of emergency cases and NGOs is manual, slow, and unreliable.
- Many donors are hesitant due to misuse of funds or fake campaigns.
- Existing platforms do not allow donors to become recipients during emergencies.
- There is a lack of an integrated digital system for monitoring donations, approvals, and fund allocation efficiently.

1.2 Objectives:

- To develop a **secure and transparent platform** connecting donors, NGOs, and emergency recipients.
- To enable **dual-role registration**, allowing donors to request help in emergencies.
- To implement an **admin verification system** for NGOs and recipient documents.
- To provide an **AI-assisted suggestion system** for fund distribution and matching.
- To build an **interactive dashboard** for admins to manage donations, cases, and volunteers efficiently.
- To ensure that every contribution, no matter how small, creates a measurable social impact.

1.3 Summary:

The **Fund4Life** project is a unique and socially impactful initiative that integrates technology, community participation, and transparency to create a **reliable digital donation ecosystem**. Unlike conventional crowdfunding systems that merely act as intermediaries between donors and recipients, Fund4Life builds a **mutual trust network** where donors, emergency recipients, and NGOs interact in a transparent and verifiable environment.

The project's central idea revolves around creating a **community-based financial assistance model**, where individuals can contribute a small fixed amount—starting from just ₹100 per month—toward helping others in medical or financial emergencies. In contrast to typical donation platforms, Fund4Life ensures that every donor also has the **option to request help** in times of personal emergencies, promoting a **reciprocal and supportive culture** within the community.

The system introduces a **dual-role mechanism**, allowing a user to act both as a donor and a recipient. This makes the platform inclusive, empowering all participants equally. Emergency recipients can upload **medical documents, photographs, and case details**, which are securely stored and verified by the admin before being made visible for public donations. The **admin verification process** ensures authenticity, prevents fraudulent cases, and enhances donor confidence.

From a technological perspective, Fund4Life is designed with a **Flask-based backend** to handle server operations, API management, and database integration, while the **React-based frontend and Admin Dashboard** provide an intuitive and interactive user experience. The admin panel manages critical operations such as **NGO approvals, case verification, fund distribution, and volunteer management**. Real-time synchronization between backend and frontend through APIs ensures that data is updated instantly and accurately.

CHAPTER 2

LITERATURE SURVEY

The **literature survey** provides an overview of existing donation and crowdfunding systems and highlights the innovations that differentiate the **Fund4Life** platform. This section evaluates the strengths and limitations of current solutions, followed by a detailed explanation of how Fund4Life addresses these challenges through its proposed system. Online donation platforms and crowdfunding systems have become increasingly popular as people seek quick and convenient ways to contribute to charitable causes. However, despite the technological advancements, many platforms continue to face issues such as **lack of transparency, poor verification mechanisms, delayed fund distribution, and low donor engagement**. These limitations underline the necessity for a secure, transparent, and community-oriented system — a gap that Fund4Life aims to fill.

2.1 Existing System:

Limitations of Existing Systems:

➤ **Lack of Transparency:**

Many existing systems do not provide donors with clear information on how their funds are used. Once the donation is made, tracking the end-use of funds becomes difficult, reducing donor trust and participation.

➤ **Weak Verification Mechanisms:**

Verification of recipients and uploaded medical or NGO documents is often manual and insufficient. This leads to the possibility of fraudulent or duplicate campaigns, resulting in donor hesitation.

➤ **No Dual-Role Flexibility:**

In existing systems, donors and recipients function separately. A person who donates regularly cannot request help during emergencies, reducing inclusivity and community bonding.

➤ **High Platform Charges and Delayed Disbursal:**

Many platforms deduct significant service fees, reducing the amount that reaches the recipient. Additionally, fund disbursal often takes days or weeks, delaying emergency assistance.

➤ **Limited Interaction Between Users:**

Donors have minimal engagement with recipients or NGOs beyond the donation itself. There is no direct communication or community-building feature

2.2 Proposed System:

Key Features and Improvements:

➤ **Dual-Role User System:**

Fund4Life introduces a unique dual-role concept where users can act as both donors and recipients. This encourages community participation and ensures that every member has the right to seek help when needed.

➤ **Admin Verification Mechanism:**

Every registration by an NGO or emergency recipient undergoes strict verification by the admin. Only verified cases are made visible to donors. This prevents fraudulent activities and builds donor confidence.

➤ **Automated Fund Distribution:**

The platform includes a smart allocation module, which distributes collected funds to verified cases based on urgency and availability. This ensures timely assistance for genuine emergencies.

➤ **Real-Time Tracking and Transparency:**

All donations, approvals, and fund disbursements are tracked in real time. Donors can view the status of their contributions, ensuring complete transparency and accountability.

➤ **Volunteer-Based Verification:**

Registered volunteers assist in verifying on-ground details of emergency cases and NGOs, improving the reliability of information and community engagement.

➤ **Secure Authentication and OTP Verification:**

The system ensures user authenticity through OTP-based login and verification, minimizing the chances of fake profiles or misuse.

➤ **Interactive Admin Dashboard:**

The admin dashboard, built using React and MUI, provides comprehensive control to manage NGOs, donations, emergency requests, and volunteers. It includes analytical visualizations for better decision-making.

CHAPTER 3

SYSTEM REQUIREMENTS

The **System Requirements** section defines the hardware, software, and operational needs necessary to implement the **Fund4Life** platform effectively. It specifies both **functional and non-functional aspects** of the project, ensuring the system is efficient, secure, and scalable. The Fund4Life application is designed as a **full-stack web system** that integrates donor management, emergency aid requests, NGO verification, and fund distribution functionalities into a single, unified platform. The requirements mentioned below provide the foundation for ensuring that the system operates reliably and performs optimally under various conditions.

3.1 Introduction:

The primary goal of defining system requirements is to outline the **technical and functional framework** for Fund4Life. These requirements describe what the system needs to achieve, the technologies used to implement it, and the expected performance standards.

Fund4Life operates as a **web-based application** that connects three major user groups — **Donors, Emergency Recipients, and Administrators (Admins)**. The Admin Dashboard enables case verification, fund allocation, and NGO approval, while donors can make secure contributions and track the impact of their donations in real time.

3.2 Software and Hardware Requirement:

Software Requirements:

- Operating System: Windows / Linux
- Backend Language: Python
- Database: MySQL
- IDE: VS Code

Hardware Requirements:

- Processor: Minimum Intel i3, Recommended Intel i5 or above
- RAM: Minimum 4 GB, Recommended 8 GB or more
- Internet: Required for AI API calls, cloud integrations, and external workflow services

3.3 Functional and Non-Functional Requirements:

Functional Requirements:

Functional requirements define what the system must perform to achieve its objectives. Fund4Life includes the following key functionalities:

1. **User Registration and Login:**
Users can register as donors or emergency recipients. OTP verification ensures secure access.
2. **Dual-Role User Management:**
Donors can switch roles to request help in emergencies, enabling inclusivity.
3. **Donation Module:**
Users can donate specific amounts securely. All transactions are recorded and viewable on dashboards.
4. **Recipient Case Submission:**
Recipients upload documents (medical reports, photos, bills) for verification by the admin.
5. **Admin Approval System:**
Admins verify NGO applications, recipient cases, and donations before approving fund distribution.
6. **Fund Distribution and Tracking:**
Approved cases receive allocated funds, and donors can track how their contributions are used.
7. **Volunteer Management System:**
Admins can register and manage volunteers for case verification and coordination.
8. **NGO Registration and Monitoring:**
NGOs register through the system and receive admin approval to participate in fund disbursal.
9. **Dashboard and Analytics:**
Admin Dashboard provides insights into total donations, active cases, NGO statuses, and fund usage statistics.
10. **Notifications and Updates:**
Email or SMS alerts are sent to donors, NGOs, and recipients for important updates like approval, fund transfer, or case verification.

Non-Functional Requirements:

Non-functional requirements specify how well the system performs its operations rather than what it does.

1. Performance:

The system must respond to user actions within seconds, even during high load. API calls and dashboard data retrieval should be optimized.

2. Security:

Sensitive data such as user credentials and documents must be encrypted. JWT tokens and OTP verification prevent unauthorized access.

3. Reliability:

The system should be reliable and available 24/7, ensuring no data loss or transaction failure.

4. Scalability:

Architecture must allow expansion, supporting increased users, NGOs, and donation cases without system degradation.

5. Usability:

User interfaces should be intuitive and easy to navigate for all age groups.

6. Maintainability:

Code structure must be modular, allowing easy updates and debugging.

7. Portability:

The system should be deployable across multiple platforms (Windows, Linux, Cloud environments).

8. Transparency:

All donation activities must be traceable, ensuring full accountability between donors and recipients.

9. Data Integrity:

Information stored in the database should remain consistent, accurate, and protected against corruption or unauthorized modification.

3.4. Other Requirements

Beyond hardware and software, Fund4Life also includes several **operational and supportive requirements** that ensure smooth performance and trustworthiness.

1. **Backup and Recovery:**

Automatic daily backups of the database to prevent data loss in case of system failure.

2. **Audit Trails:**

Every admin action, approval, and fund transaction is logged for accountability and traceability.

3. **API Security:**

All backend API routes are secured using authentication tokens and HTTPS protocols.

4. **Accessibility:**

The system must be accessible from both desktop and mobile browsers for convenience.

5. **Integration Support:**

Future compatibility with AI services or payment gateways such as Razorpay or Paytm for automation.

6. **Data Analytics & Reporting:**

Analytical graphs in the admin dashboard to monitor donation patterns, user engagement, and fund usage.

7. **Email & Notification Service:**

Automated notifications for approvals, fund receipts, and emergency alerts to users and NGOs.

8. **Documentation:**

Complete technical documentation for developers and user manuals for admins and donors.

3.5 Summary:

The system requirements defined above form the foundation for developing a **secure, transparent, and community-oriented donation management platform**. By combining modern technologies like **Flask, React, and MySQL**, Fund4Life ensures **high performance, scalability, and reliability** while addressing real-world challenges in the donation ecosystem.

The structured functional modules, coupled with robust non-functional attributes such as security, transparency, and usability, guarantee a seamless and trustworthy user experience. These requirements serve as the blueprint for implementing Fund4Life successfully and ensuring it achieves its ultimate goal — **to make financial help accessible, verifiable, and impactful for everyone in need**.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction:

System Design is the process of defining the **architecture, components, modules, interfaces, and data flow** of a system to meet specific requirements. For **Fund4Life**, system design ensures that every module — donor registration, emergency recipient verification, NGO approval, and fund distribution — is integrated seamlessly within a secure and transparent ecosystem.

The design phase bridges the gap between theoretical requirements and practical implementation. It defines how the **Flask backend**, **React frontend**, and **MySQL database** interact, ensuring smooth communication between all components. This phase focuses on the **logical, physical, and architectural structure** of the platform.

4.2 Architectural Diagram:

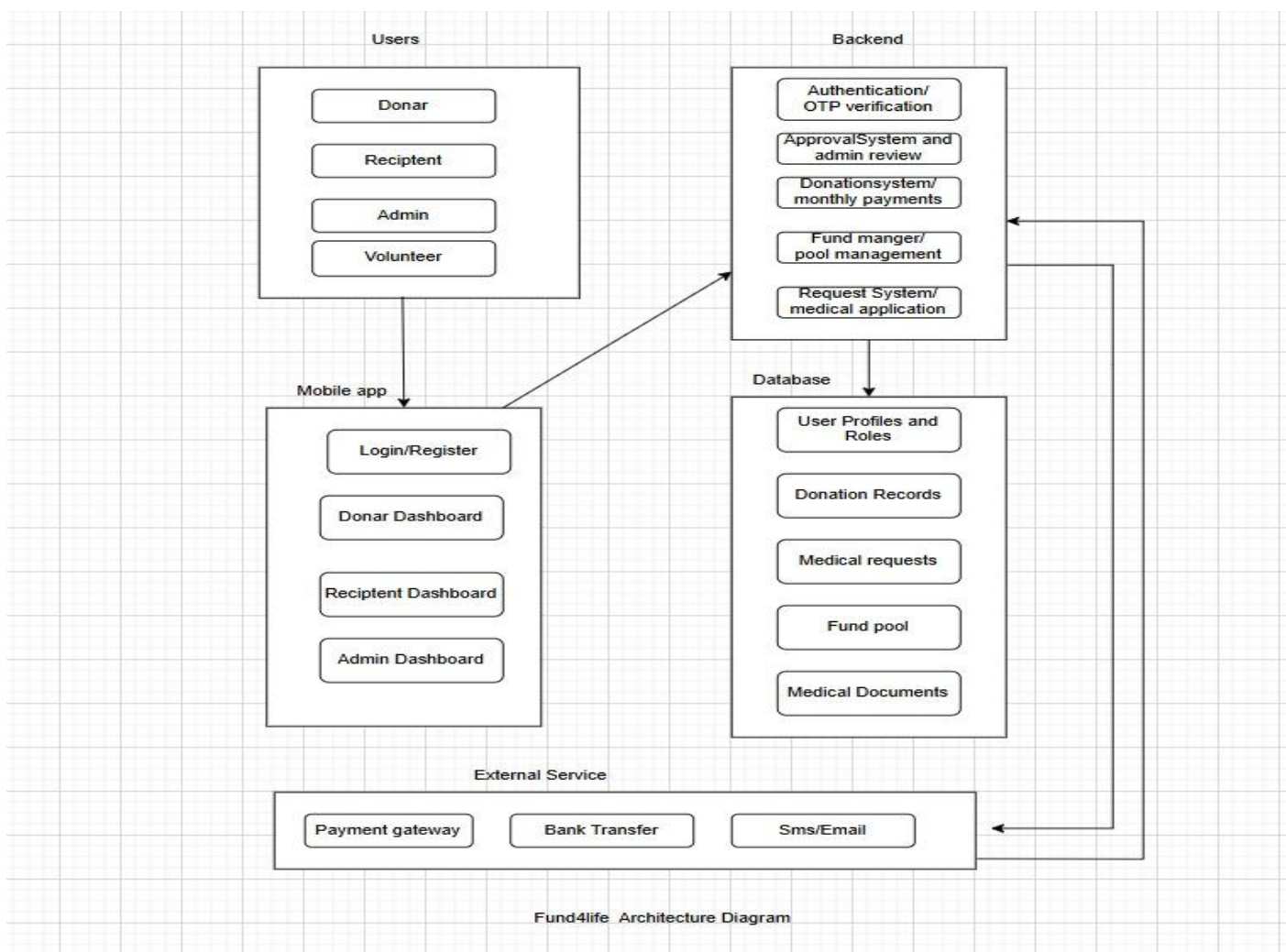


Fig – 4.2

4.3 Data Flow Diagram/UML Diagram

4.3.1 Use Case Diagram:



Fig – 4.3.1

4.3.2 Class Diagram:

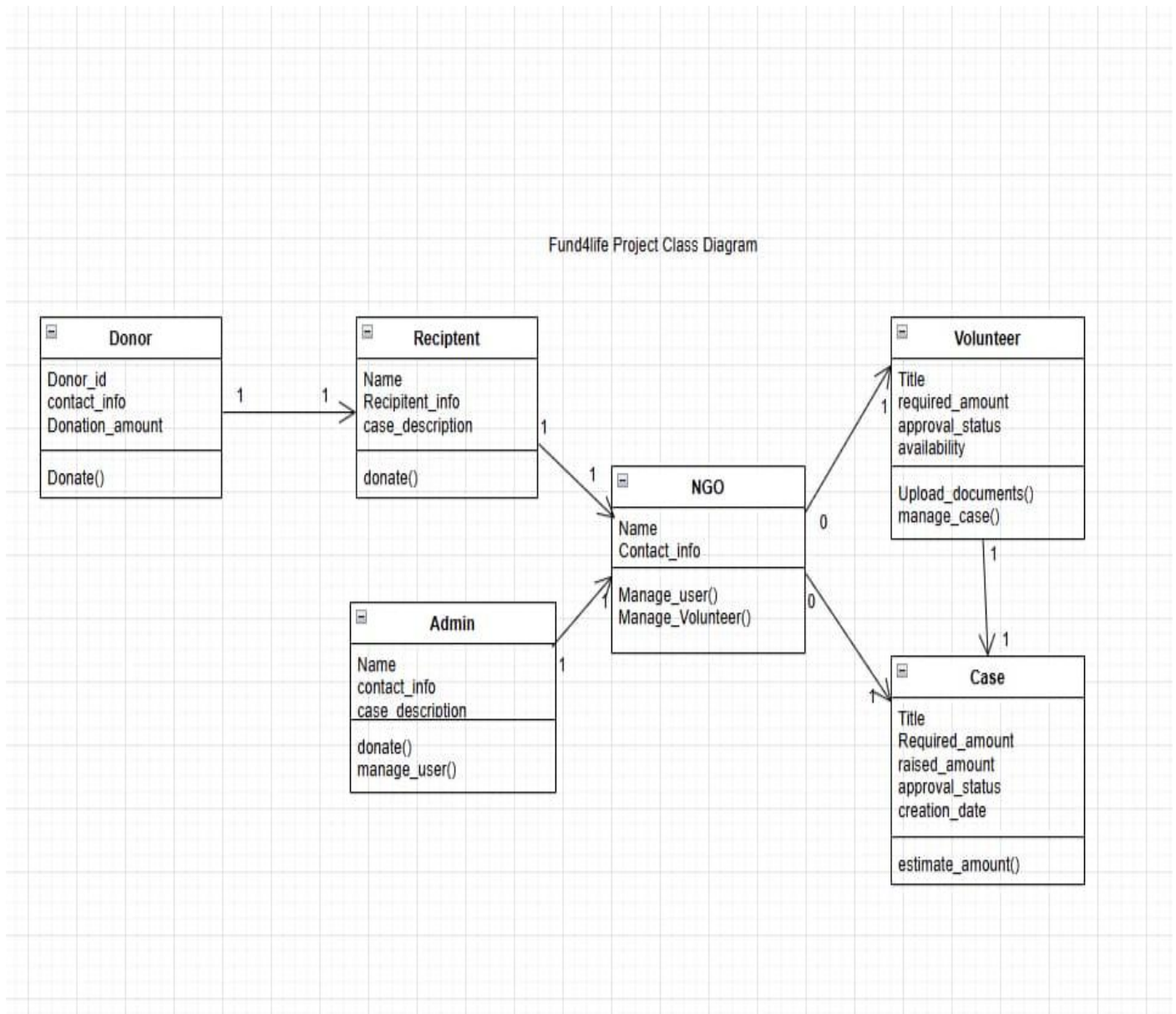


Fig – 4.3.2

4.3.3 Sequence Diagram:

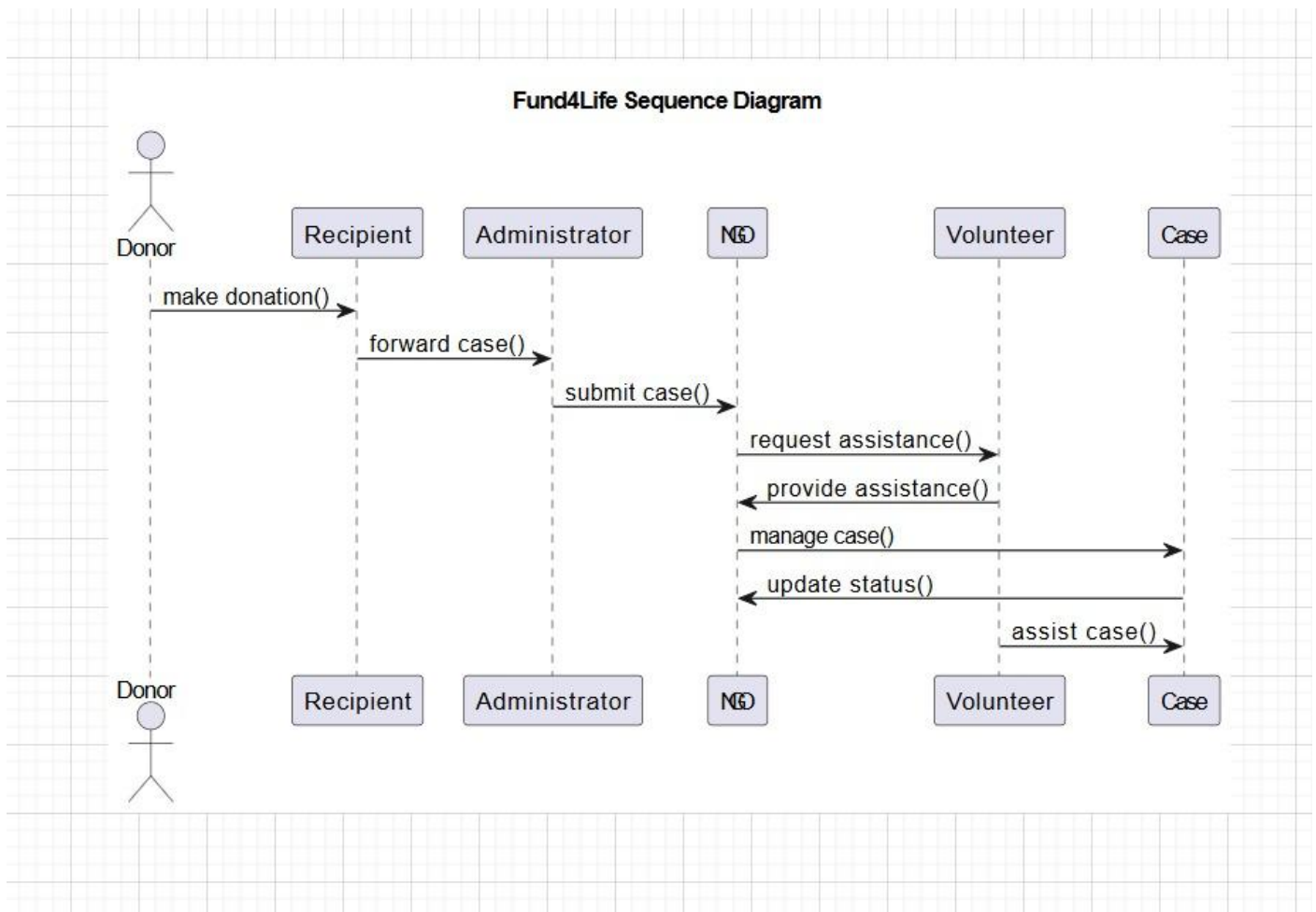


Fig – 4.3.3

4.3.4 Activity Diagram:

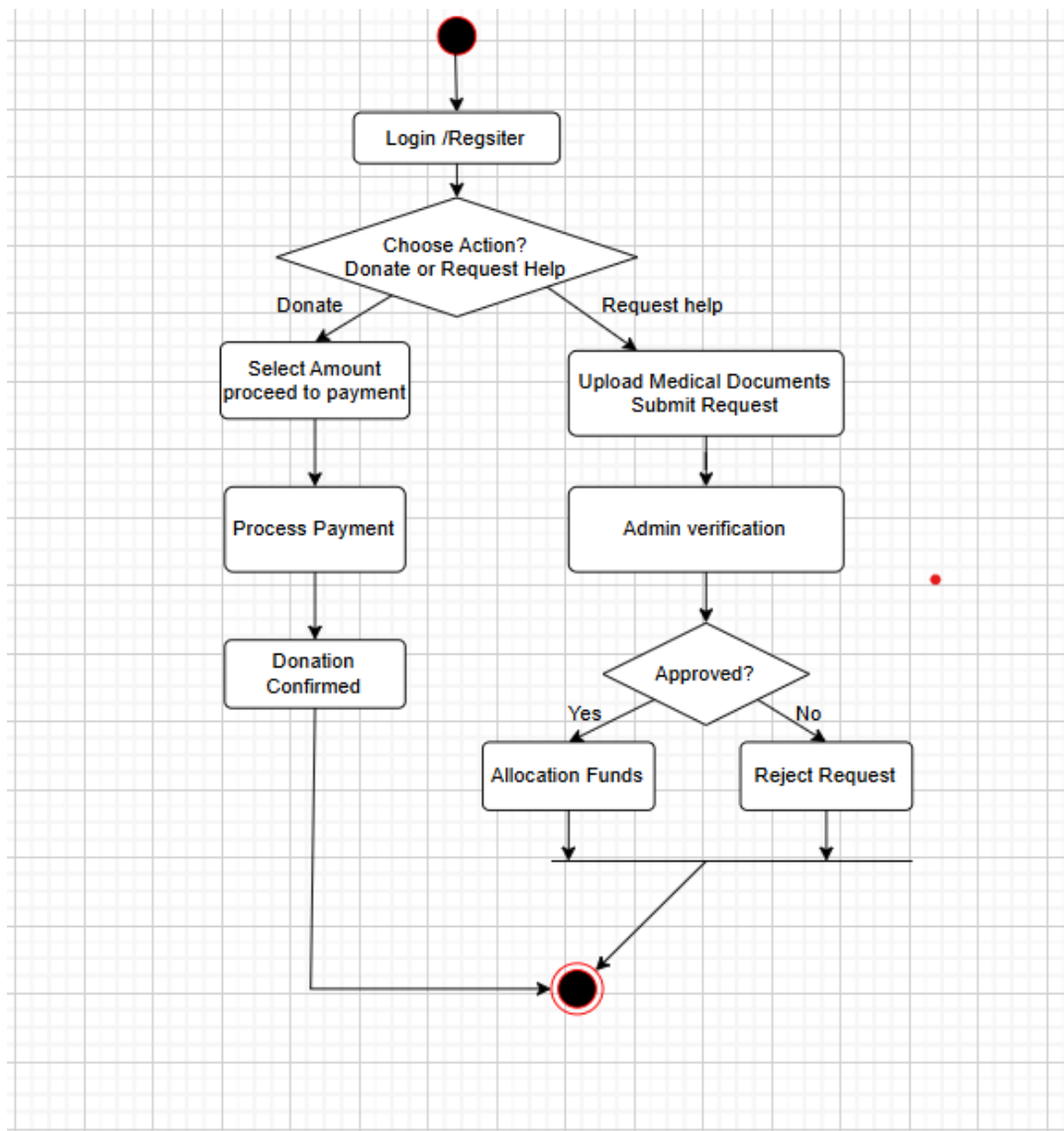


Fig – 4.3.4

4.4 Summary:

The system design phase ensures that **Fund4Life** operates as a structured, modular, and interconnected application. The architecture provides **security, scalability, and transparency**, while UML diagrams define the logical workflow between users, backend, and database components. This design forms the backbone for the next stage — **Implementation**.

CHAPTER 5

IMPLEMENTATION

5.1 Algorithms:

The implementation of Fund4Life relies on structured algorithms for key processes:

1. User Registration and Login Algorithm
 - Input: Email, Password, OTP
 - Process: Verify OTP → Store user → Generate token
 - Output: “Registration Successful”
2. Donation Algorithm
 - Input: Donation amount, user ID
 - Process: Validate donor → Record transaction → Update total funds
 - Output: “Donation Successful”
3. Fund Allocation Algorithm
 - Input: Verified recipient list
 - Process: Match donation pool with urgent cases → Allocate proportionally
 - Output: Updated records in DB and recipient notifications
4. Verification Algorithm
 - Input: Recipient/NGO data and documents
 - Process: Admin validates documents → Changes status to Approved/Rejected
 - Output: Case visible to donors or flagged for review

5.2 Architectural Components:

Frontend (React + MUI)

- Components: Login Page, Dashboard, Donation Page, Recipient Upload Form, Admin Panel.
- Uses Axios for API communication.

Backend (Flask)

- Routes: /register, /login, /donate, /approve, /funds.
- Manages authentication, data validation, and fund logic.

Database (MySQL/SQLite)

- Tables: users, donations, recipients, ngos, funds.
- Stores encrypted and relational data.

Security Layer

- Implements JWT for session tokens and OTP verification for login.

Admin Dashboard

- Displays total donations, approved cases, NGO status, and analytics charts.

5.3 Feature Extraction:

In the Fund4Life system, feature extraction refers to identifying and processing the key information provided by users to enable functionalities such as verification, fund allocation, and report generation. The system collects and refines useful features from various modules, ensuring data accuracy, transparency, and efficiency.

1. User Data Extraction

During registration, the system extracts essential user details such as name, email, phone number, role type (donor, recipient, or NGO), and stores them securely in the database. These features are used for authentication, OTP verification, and role-based access within the platform.

2. Document Features

When recipients upload medical documents and photos, the system captures features like file type, size, upload time, and description. This helps the admin verify case authenticity and ensures that only valid emergency requests are approved.

3. Donation Features

For every donation, the system extracts key transaction features including donor ID, donation amount, date/time, and recipient or NGO ID. These features are used for fund tracking, real-time updates, and generating summary reports for transparency.

4. NGO and Volunteer Features

NGOs and volunteers provide specific details such as organization name, area of service, and approval status. These extracted features help in managing the workflow of fund allocation, case verification, and monitoring.

5. Security and Validation

All extracted data undergoes validation and encryption before being stored. Invalid or duplicate entries are filtered out to maintain data accuracy and system security.

5.4. Packages/Libraries Used:

The **Fund4Life** system is developed using a combination of **frontend** and **backend** technologies. Each side plays a vital role in ensuring smooth functionality, secure data handling, and an interactive user experience.

A. Backend (Flask – Python)

1. Flask

- A lightweight Python web framework used to develop the backend.
- Handles API creation, routing, and server-side logic.

2. Flask-SQLAlchemy

- ORM tool for managing database models and performing CRUD operations using Python code instead of SQL queries.

3. Flask-Mail / SMTPLib

- Used to send OTP verification emails and user notifications during registration and login.

4. Flask-CORS

- Enables secure communication between the React frontend and Flask backend by allowing cross-origin requests.

5. MySQL Connector

- Establishes a secure connection with the MySQL database for storing and retrieving data like donations, NGOs, and recipients.

6. Werkzeug Security

- Provides password hashing and authentication utilities, ensuring user credentials are safely stored.

7. JWT (JSON Web Token)

- Used for authentication and access control to protect admin and user routes.

Frontend (React JS – JavaScript)

1. React JS

- A JavaScript library used for building dynamic and responsive interfaces.
- Enables single-page application functionality for smooth navigation.

2. Material UI (MUI)

- Provides pre-designed UI components like tables, buttons, modals, and cards for a professional, consistent layout.

3. Axios

- Used for API communication between frontend and backend via GET and POST requests.

4. React Router DOM

- Manages navigation and protected routes, ensuring secure transitions between pages like login, dashboard, and NGO approval.

5. React Context API

- Handles global state management (e.g., storing authentication and user details) across different components.

5.5 Code:

Fund4life_App.py:

```
from flask import Flask, render_template

from extensions import db

from models import donor, ngo, emergency_case, donation

from auth import auth_bp

from donor import donor_bp

from admin.routes import admin_bp

from ngo import ngo_bp

from volunteer_auth import volunteer_auth_bp

from volunteer import volunteer_bp


def create_app():

    app = Flask(__name__)

    app.config['SECRET_KEY'] = 'supersecretkey'

    app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///fund4life.db'

    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False


    # Initialize SQLAlchemy with app

    db.init_app(app)


    # Register blueprints for modular route management

    app.register_blueprint(auth_bp)

    app.register_blueprint(donor_bp)

    app.register_blueprint(admin_bp)

    app.register_blueprint(ngo_bp)

    app.register_blueprint(volunteer_auth_bp)
```

```

app.register_blueprint(volunteer_bp)

# Root route for homepage
@app.route('/')
def index():
    return render_template('home.html')

# Create tables if not exist
with app.app_context():
    db.create_all()

return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)

```

Fund4life_Routes.py

```

from flask import Blueprint, render_template, redirect, url_for, flash, request, session,
current_app

from extensions import db

from models.ngo import NGO

from models.emergency_case import EmergencyCase

from models.donor import Donor

from models.donation import Donation

from models.verification_report import VerificationReport

import os

from flask import send_from_directory

```

```
admin_bp = Blueprint('admin', __name__, url_prefix='/admin')
```

```
ADMIN_EMAIL = 'admin@fund4life.com'
```

```
ADMIN_PASSWORD = 'admin123'
```

```
def admin_required(f):
```

```
    from functools import wraps
```

```
    @wraps(f)
```

```
    def decorated(*args, **kwargs):
```

```
        if not session.get('admin_logged_in'):
```

```
            flash('Admin login required.', 'danger')
```

```
            return redirect(url_for('admin.login'))
```

```
        return f(*args, **kwargs)
```

```
    return decorated
```

```
@admin_bp.route('/login', methods=['GET', 'POST'])
```

```
def login():
```

```
    if request.method == 'POST':
```

```
        email = request.form['email']
```

```
        password = request.form['password']
```

```
        if email == ADMIN_EMAIL and password == ADMIN_PASSWORD:
```

```
            session['admin_logged_in'] = True
```

```
            flash('Admin login successful.', 'success')
```

```
            return redirect(url_for('admin.dashboard'))
```

```
        flash('Invalid admin credentials.', 'danger')
```

```
    return render_template('admin/admin_login.html')
```

```
@admin_bp.route('/logout')
```

```
def logout():
```

```
    session.pop('admin_logged_in', None)
```

```
    flash('Logged out.', 'info')
```

```
    return redirect(url_for('admin.login'))
```

```
@admin_bp.route('/dashboard')
```

```
@admin_required
```

```
def dashboard():
```

```
    total_donations = db.session.query(db.func.sum(Donation.amount)).scalar() or 0
```

```
    donors = Donor.query.count()
```

```
    ngos = NGO.query.count()
```

```
    cases = EmergencyCase.query.count()
```

```
    return render_template('admin/dashboard.html', total_donations=total_donations, donors=donors,  
ngos=ngos, cases=cases)
```

```
@admin_bp.route('/ngo_approvals', methods=['GET', 'POST'])
```

```
@admin_required
```

```
def ngo_approvals():
```

```
    ngos = NGO.query.all()
```

```
    if request.method == 'POST':
```

```
        ngo_id = request.form['ngo_id']
```

```
        action = request.form['action']
```

```
        ngo = NGO.query.get(ngo_id)
```

```
        if ngo:
```

```
            ngo.is_approved = (action == 'approve')
```

```
            db.session.commit()
```

```

        flash(f"NGO {'approved' if action == 'approve' else 'rejected'}.", 'success')

    return render_template('admin/ngo_approvals.html', ngos=ngos)

@admin_bp.route('/cases', methods=['GET', 'POST'])
@admin_required
def cases():
    all_cases = EmergencyCase.query.all()
    ngos = NGO.query.filter_by(is_approved=True).all()

    if request.method == 'POST':
        case_id = request.form['case_id']
        assigned_ngo = request.form.get('assigned_ngo')
        action = request.form['action']
        case = EmergencyCase.query.get(case_id)

        if case:
            if action == 'assign' and assigned_ngo:
                case.assigned_ngo = int(assigned_ngo)
                case.status = 'Assigned'
            elif action == 'approve':
                case.status = 'Approved'
            elif action == 'reject':
                case.status = 'Rejected'

            db.session.commit()

            flash('Case updated.', 'success')

    return render_template('admin/cases.html', cases=all_cases, ngos=ngos)

@admin_bp.route('/approve_funds/<int:case_id>', methods=['POST'])
@admin_required

```

```

def approve_funds(case_id):

    case = EmergencyCase.query.get_or_404(case_id)

    amount = request.form.get('amount', type=float)

    if amount is None or amount <= 0:

        flash('Please enter a valid amount.', 'danger')

        return redirect(url_for('admin.cases'))

    # Update case amount and mark funds as released

    case.amount = amount

    case.status = 'Funds Released'

    db.session.commit()

    flash('Funds approved and successfully sent to recipient.', 'success')

    return redirect(url_for('admin.cases'))


@admin_bp.route('/verification_reports')
@admin_required
def verification_reports():

    reports = VerificationReport.query.all()

    return render_template('admin/verification_reports.html', reports=reports)


@admin_bp.route('/verification_report/<int:report_id>')
@admin_required
def verification_report_details(report_id):

    report = VerificationReport.query.get_or_404(report_id)

    return render_template('admin/verification_report_details.html', report=report)


@admin_bp.route('/uploads/<path:filename>')
@admin_required

```



```
def uploaded_file(filename):

    uploads = os.path.join(current_app.root_path, 'static', 'verification_uploads')

    return send_from_directory(uploads, filename)
```

Donor.py:

```
import os from flask import Blueprint, render_template, redirect, url_for, flash, request, session, current_app

from werkzeug.utils import secure_filename

from extensions import db

from models.donor import Donor

from models.donation import Donation

from models.emergency_case import EmergencyCase


donor_bp = Blueprint('donor', __name__, url_prefix='/donor')


def donor_required(f):

    from functools import wraps

    @wraps(f)

    def decorated(*args, **kwargs):

        if 'donor_id' not in session:

            flash('Please log in as donor.', 'danger')

            return redirect(url_for('auth.login'))

        return f(*args, **kwargs)

    return decorated


@donor_bp.route('/dashboard')

@donor_required

def dashboard():

    donor = Donor.query.get(session['donor_id'])
```

```

donations = Donation.query.filter_by(donor_id=donor.id).all()

cases = EmergencyCase.query.filter_by(donor_id=donor.id).all()

total_donated = sum([d.amount for d in donations])

return render_template('donor/dashboard.html', donor=donor, donations=donations, cases=cases,
total_donated=total_donated)

```

```
@donor_bp.route('/donate', methods=['POST'])
```

```
@donor_required
```

```
def donate():
```

```
    amount = float(request.form['amount'])
```

```
    donation = Donation(donor_id=session['donor_id'], amount=amount)
```

```
    db.session.add(donation)
```

```
    db.session.commit()
```

```
    flash('Thank you for your donation!', 'success')
```

```
    return redirect(url_for('donor.dashboard'))
```

```
@donor_bp.route('/emergency_request', methods=['POST'])
```

```
@donor_required
```

```
def emergency_request():
```

```
    description = request.form['description']
```

```
    amount_requested = float(request.form['amount_requested'])
```

```
    document = request.files.get('document')
```

```
    filename = None
```

```
    if document:
```

```
        filename = secure_filename(document.filename)
```

```
        doc_path = os.path.join(current_app.root_path, 'static', 'documents')
```

```
        os.makedirs(doc_path, exist_ok=True)
```

```
        document.save(os.path.join(doc_path, filename))
```

```

case = EmergencyCase(
    donor_id=session['donor_id'],
    description=description,
    amount_requested=amount_requested,
    status='Pending',
    document_filename=filename
)

db.session.add(case)

db.session.commit()

flash('Emergency case submitted.', 'success')

return redirect(url_for('donor.dashboard'))

```

Volunteer.py:

```

import os

from flask import Blueprint, render_template, redirect, url_for, flash, request, session, current_app

from werkzeug.utils import secure_filename

from extensions import db

from models.volunteer import Volunteer

from models.emergency_case import EmergencyCase

from models.verification_report import VerificationReport


volunteer_bp = Blueprint('volunteer', __name__, url_prefix='/volunteer')

```

```

def volunteer_required(f):

    from functools import wraps

    @wraps(f)
    def decorated(*args, **kwargs):

        if 'volunteer_id' not in session:

            flash('Please log in as volunteer.', 'danger')

            return redirect(url_for('volunteer_auth.login'))

        return f(*args, **kwargs)

    return decorated


@volunteer_bp.route('/dashboard')

@volunteer_required
def dashboard():

    volunteer = Volunteer.query.get(session['volunteer_id'])

    assigned_cases = EmergencyCase.query.filter_by(assigned_volunteer=volunteer.id).all()

    case_details = []

    for case in assigned_cases:

        # Fetch related verification report if any

        verification_report = VerificationReport.query.filter_by(case_id=case.id,
volunteer_id=volunteer.id).order_by(VerificationReport.submitted_at.desc()).first()

        verification_status = "Not Submitted"

        if verification_report:

            verification_status = "Submitted"

        case_details.append({

            'case': case,

            'verification_status': verification_status,

```

```

        'verification_report': verification_report
    })

    return render_template('volunteer/dashboard.html', volunteer=volunteer, case_details=case_details)

@volunteer_bp.route('/case/<int:case_id>', methods=['GET', 'POST'])
@volunteer_required
def submit_verification(case_id):
    volunteer = Volunteer.query.get(session['volunteer_id'])
    case = EmergencyCase.query.get_or_404(case_id)

    if case.assigned_volunteer != volunteer.id:
        flash('Unauthorized access to case.', 'danger')
        return redirect(url_for('volunteer.dashboard'))

    if request.method == 'POST':
        live_photo = request.files.get('live_photo')
        verified_docs = request.files.get('verified_docs')
        comments = request.form.get('comments')

        upload_folder = os.path.join(current_app.root_path, 'static', 'verification_uploads')
        os.makedirs(upload_folder, exist_ok=True)

        live_photo_filename = None
        verified_docs_filename = None

        if live_photo:

```

```

live_photo_filename = secure_filename(live_photo.filename)

live_photo.save(os.path.join(upload_folder, live_photo_filename))


if verified_docs:

    verified_docs_filename = secure_filename(verified_docs.filename)

    verified_docs.save(os.path.join(upload_folder, verified_docs_filename))


report = VerificationReport(

    case_id=case.id,

    volunteer_id=volunteer.id,

    live_photo_filename=live_photo_filename,

    verified_docs_filename=verified_docs_filename,

    comments=comments

)

db.session.add(report)

case.status = 'Verified by Volunteer'

db.session.commit()


flash('Verification report submitted.', 'success')

return redirect(url_for('volunteer.dashboard'))


return render_template('volunteer/submit_verification.html', case=case)

```

Admin Dashboard:

```

{% extends 'base.html' %}

{% block content %}

<h2>Admin Dashboard</h2>

```

```

<ul>

    <li>Total Donations: ₹{{ total_donations }}</li>

    <li>Donors Registered: {{ donors }}</li>

    <li>NGOs Registered: {{ ngos }}</li>

    <li>Emergency Cases Submitted: {{ cases }}</li>

</ul>

<a href="{{ url_for('admin.ngo_approvals') }}">NGO Approvals</a> |

<a href="{{ url_for('admin.cases') }}">Manage Emergency Cases</a>

{% endblock %}

```

Donor Dashboard:

```

{% extends 'base.html' %}

{% block content %}

<h2>Welcome, {{ donor.name }}</h2>

<p>Phone: {{ donor.phone }}</p> <!-- Show phone number -->


<h3>Dashboard</h3>

<p>Total Donated: ₹{{ total_donated }}</p>


<h4>Your Donations</h4>

<ul>

    {% for d in donations %}

        <li>₹{{ d.amount }} on {{ d.timestamp.strftime('%Y-%m-%d') }}</li>

    {% else %}

        <li>No donations yet.</li>

    {% endfor %}

</ul>

```

<h4>Add a Donation</h4>

<form method="POST" action="{{ url_for('donor.donate') }}">

<input type="number" name="amount" step="0.01" placeholder="Amount (INR)" required>

<button type="submit">Donate</button>

</form>

<h4>Submit Emergency Request</h4>

<form method="POST" action="{{ url_for('donor.emergency_request') }}" enctype="multipart/form-data">

<textarea name="description" placeholder="Describe your emergency" required></textarea>

<input type="number" name="amount_requested" step="0.01" placeholder="Requested Amount" required>

<input type="file" name="document">

<button type="submit">Submit Request</button>

</form>

<h4>Your Emergency Cases</h4>

{% for c in cases %}

{{ c.description }} | ₹{{ c.amount_requested }} | Status: {{ c.status }}

{% if c.document_filename %}

| Document

{% endif %}

{% else %}


```

    <li>No emergency cases submitted.</li>

{% endfor %}

</ul>

{% endblock %}

```

NGO Dashboard:

```

{% extends 'base.html' %}

{% block content %}

<h2>Welcome, {{ ngo.name }}</h2>


<h3>Cases Assigned to You</h3>


{% if case_details %}

<table border="1" cellpadding="6">

  <thead>

    <tr>

      <th>Case Description</th>

      <th>Amount Requested</th>

      <th>Assigned Volunteer</th>

      <th>Verification Status</th>

      <th>Amount Released</th>

      <th>Actions</th>

    </tr>

  </thead>

  <tbody>

    {% for item in case_details %}

      <tr>

```

```

<td>{{ item.case.description }}</td>

<td>₹{{ item.case.amount_requested }}</td>

<td>{{ item.volunteer.name if item.volunteer else 'Not Assigned' }}</td>

<td>{{ item.verification_status }}</td>

<td>₹{{ item.case.amount_released if item.case.amount_released else '0' }}</td>

<td>

    <!-- For example, links to assign volunteer, update status, upload docs etc. -->

    <a href="{{ url_for('ngo.assign_volunteer', case_id=item.case.id) }}">Assign Volunteer</a> |

    <a href="{{ url_for('ngo.update_case_status', case_id=item.case.id) }}">Update Status</a>

</td>

</tr>

{% endfor %}

</tbody>

</table>

{% else %}

<p>No cases assigned to your NGO yet.</p>

{% endif %}

<a href="{{ url_for('ngo.logout') }}">Logout</a>

{% endblock %}

```

Volunteer Dashboard:

```

{% extends 'base.html' %}

{% block content %}

<h2>Welcome, {{ volunteer.name }}</h2>

<h3>Your Assigned Cases</h3>

```

```

{% if case_details %}

<table border="1" cellpadding="6">

  <thead>

    <tr>

      <th>Description</th>

      <th>Amount Requested</th>

      <th>Status</th>

      <th>Verification Status</th>

      <th>Action</th>

    </tr>

  </thead>

  <tbody>

    {% for item in case_details %}

      <tr>

        <td>{{ item.case.description }}</td>

        <td>₹{{ item.case.amount_requested }}</td>

        <td>{{ item.case.status }}</td>

        <td>{{ item.verification_status }}</td>

        <td>

          {% if item.verification_status == 'Not Submitted' %}

            <a href="{{ url_for('volunteer.submit_verification', case_id=item.case.id) }}">Submit
Verification</a>

          {% else %}

            Completed

          {% endif %}

        </td>

      </tr>

    {% endfor %}


```

```

        </tbody>

</table>

{% else %}

<p>No cases assigned to you yet.</p>

{% endif %}

{% endblock %}

```

BASE.html:

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1" />

    <title>Fund4Life</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/styles.css') }}" />

</head>

<body>

<nav role="navigation" aria-label="Main navigation">

    <a href="{{ url_for('index') }}">Fund4Life</a>

    {% if session.get('donor_id') %}

        <a href="{{ url_for('donor.dashboard') }}">Donor Dashboard</a>

        <a href="{{ url_for('auth.logout') }}">Logout</a>

    {% elif session.get('ngo_id') %}

        <a href="{{ url_for('ngo.dashboard') }}">NGO Dashboard</a>

        <a href="{{ url_for('ngo.logout') }}">Logout</a>

    {% elif session.get('volunteer_id') %}

```

```

    <a href="{{ url_for('volunteer.dashboard') }}">Volunteer Dashboard</a>

    <a href="{{ url_for('volunteer_auth.logout') }}">Logout</a>

{% elif session.get('admin_logged_in') %}

    <a href="{{ url_for('admin.dashboard') }}">Admin Dashboard</a>

    <a href="{{ url_for('admin.logout') }}">Logout</a>

{% else %}

    <a href="{{ url_for('auth.register') }}">Register</a>

    <a href="{{ url_for('auth.login') }}">Login</a>

    <a href="{{ url_for('ngo.login') }}">NGO Login</a>

    <a href="{{ url_for('volunteer_auth.login') }}">Volunteer Login</a>

    <a href="{{ url_for('admin.login') }}">Admin Login</a>

{% endif %}

</nav>

<main role="main">

    {% for category, message in get_flashed_messages(with_categories=true) %}

        <div class="alert {{ category }}">{{ message }}</div>

    {% endfor %}

    {% block content %}{% endblock %}

</main>

</body>

</html>

```

HOME.html:

```

{% extends "base.html" %}

{% block content %}

```

```

<h1>Welcome to Fund4Life</h1>

<p>A community-driven donation and emergency assistance platform.</p>

<ul>

    <li><a href="{{ url_for('auth.register') }}">Register as Donor</a></li>

    <li><a href="{{ url_for('auth.login') }}">Donor Login</a></li>

    <li><a href="{{ url_for('ngo.register') }}">Register as NGO</a></li>

    <li><a href="{{ url_for('ngo.login') }}">NGO Login</a></li>

    <li><a href="{{ url_for('volunteer_auth.register') }}">Register as Volunteer</a></li>

    <li><a href="{{ url_for('volunteer_auth.login') }}">Volunteer Login</a></li>

    <li><a href="{{ url_for('admin.login') }}">Admin Login</a></li>

</ul>

{% endblock %}

```

Ngo.py:

```

from extensions import db

class NGO(db.Model):

    __tablename__ = 'ngo'

    id = db.Column(db.Integer, primary_key=True)

    name = db.Column(db.String(120), nullable=False)

    email = db.Column(db.String(120), unique=True, nullable=False)

```

```

description = db.Column(db.String(300), nullable=False)

password = db.Column(db.String(200), nullable=False) # Password field for hashed password

is_approved = db.Column(db.Boolean, default=False)

assigned_cases = db.relationship('EmergencyCase', backref='assigned_ngo_obj', lazy=True,
foreign_keys='EmergencyCase.assigned_ngo')

```

`Emergency_case.py:`

```

from extensions import db

```

```

class EmergencyCase(db.Model):

    __tablename__ = 'emergency_case'

    id = db.Column(db.Integer, primary_key=True)

    donor_id = db.Column(db.Integer, db.ForeignKey('donor.id'), nullable=False)

    description = db.Column(db.String(500), nullable=False)

    amount_requested = db.Column(db.Float, nullable=False)

    status = db.Column(db.String(20), default='Pending')

    assigned_ngo = db.Column(db.Integer, db.ForeignKey('ngo.id'), nullable=True)

    assigned_volunteer = db.Column(db.Integer, db.ForeignKey('volunteer.id'), nullable=True) # Add
this field

    created_at = db.Column(db.DateTime, server_default=db.func.now())

    document_filename = db.Column(db.String(300), nullable=True)

```

`recipitent.py`

```

from extensions import db

from datetime import datetime

class Recipient(db.Model):

    __tablename__ = 'recipients'

```

```
id = db.Column(db.Integer, primary_key=True)

name = db.Column(db.String(100), nullable=False)

email = db.Column(db.String(100), unique=True, nullable=False)

password = db.Column(db.String(200), nullable=False)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
def set_password(self, password):

    # For simplicity, store plain text; later use hashing

    self.password = password
```

```
def check_password(self, password):

    return self.password == password
```


5.5 Output Screens:

[Fund4Life](#) [Register](#) [Login](#) [NGO Login](#) [Volunteer Login](#) [Admin Login](#)

Welcome to Fund4Life

A community-driven donation and emergency assistance platform.

- [Register as Donor](#)
- [Donor Login](#)
- [Register as NGO](#)
- [NGO Login](#)
- [Register as Volunteer](#)
- [Volunteer Login](#)
- [Admin Login](#)

[Fund4Life](#) [Register](#) [Login](#) [NGO Login](#) [Volunteer Login](#) [Admin Login](#)

Register as Donor

Full Name

Email

Phone Number

Password

Already registered? [Login here.](#)

Fund4LifeRegisterLoginNGO LoginVolunteer LoginAdmin Login

Donor Login

Email

Password

Login

New user? [Register here.](#)

Fund4LifeDonor DashboardLogout

Login successful.

Welcome, Krishna

Phone: 2345234576

Dashboard

Total Donated: ₹5000.0

Your Donations

- ₹5000.0 on 2025-09-20

Add a Donation

Amount (INR)

Donate

Submit Emergency Request

Describe your emergency

Your Donations

- ₹5000.0 on 2025-09-20

Add a Donation

Submit Emergency Request

No file chosen

Your Emergency Cases

- i am very sick | ₹5000.0 | Status: Funds Released | [Document](#)

[Fund4Life](#)[Register](#)[Login](#)[NGO Login](#)[Volunteer Login](#)[Admin Login](#)

Admin Login

Email:

Password:

Admin login successful.

Admin Dashboard

- Total Donations: ₹89900.0
- Donors Registered: 7
- NGOs Registered: 1
- Emergency Cases Submitted: 6

[NGO Approvals](#) | [Manage Emergency Cases](#)

NGO Approvals

Name	Email	Status	Action
ajay	ajay@gamil.com	Approved	<button>Approve</button> <button>Reject</button>

Manage Emergency Cases

Case ID	Description	Phone Number	Amount	Status	NGO Assigned	Document	Action
1	need emergency help	N/A	₹0.00	Funds Released	None	No Document	Funds Sent
2	need money for me in emergency medical	N/A	₹0.00	Funds Released	None	No Document	Funds Sent
3	emergency	N/A	₹0.00	Funds Released	None	No Document	Funds Sent
4	emrgency need	N/A	₹0.00	Under Review	None	No Document	N/A

NGO Registration

NGO Login

Welcome, ajay

Cases Assigned to You

Case Description	Amount Requested	Assigned Volunteer	Verification Status	Amount Released	Actions
need emergency help	₹5000.0	madhu	Submitted	₹0	Assign Volunteer Update Status
need money for me in emergency medical	₹40000.0	madhu	Submitted	₹0	Assign Volunteer Update Status
emergency	₹50000.0	madhu	Submitted	₹0	Assign Volunteer Update Status

Assign Volunteer to Case

Case Description: adminted in hospital due to fever

Select Volunteer:

--Choose a volunteer--

Assign Volunteer

[Back to Dashboard](#)

Update Case Status

Case Description: adminted in hospital due to fever

Select New Status:

Funds Released

Update Status

[Back to Dashboard](#)

Volunteer Registration

Name:

Email:

Phone:

Password:

Register

[Already have an account? Login](#)

Volunteer Login

Email:

Password:

Login

[Register as Volunteer](#)

Login successful.

Welcome, madhu

Your Assigned Cases

Description	Amount Requested	Status	Verification Status	Action
need emergency help	₹5000.0	Funds Released	Submitted	Completed
need money for me in emergency medical	₹40000.0	Funds Released	Submitted	Completed
emergency	₹50000.0	Funds Released	Submitted	Completed
emrgency need	₹20000.0	Under Review	Not Submitted	Submit Verification

Submit Verification for Case

Description: emrgency need

Live Photo:*

No file chosen

Verified Documents:

No file chosen

Comments:

[Back to Dashboard](#)

CHAPTER 6

SYSTEM TESTING

6.1 Introduction:

System testing is a crucial phase in software development where the complete integrated system is tested as a whole to ensure that all components work together as intended. In the **Fund4Life** project, system testing was conducted to verify the functionality, reliability, and performance of both the **frontend (React)** and **backend (Flask)** modules. The goal was to ensure that the donation platform operates smoothly, maintains data integrity, and provides a secure and user-friendly experience for all users — donors, recipients, NGOs, and the admin.

Testing involved both **manual and automated** techniques, covering functional, non-functional, and integration aspects. Each module — including **user registration, OTP verification, donation processing, case submission, admin approval, and fund distribution** — was tested thoroughly to ensure expected behavior under different conditions.

6.2 Testcases:

Test Case ID	Test Case Description	Input / Action	Expected Output	Result
TC01	User Registration with OTP	Enter valid name, email, password	OTP sent to email and account created after verification	Passed
TC02	User Login	Enter valid credentials	Redirect to dashboard	Passed
TC03	Invalid Login	Enter incorrect credentials	Display error message "Invalid username or password"	Passed
TC04	Donation Submission	Donor selects recipient and enters amount	Donation successfully recorded and updated in backend	Passed
TC05	Recipient Emergency Request	Upload medical documents and submit	Request saved and shown as "Pending Approval" to Admin	Passed
TC06	Admin Case Approval	Admin approves verified requests	Request status changes to "Approved"	Passed
TC07	NGO Registration	NGO submits registration form	Request saved in admin's NGO list for verification	Passed
TC08	Fund Distribution	Admin allocates fund to approved case	Recipient record updated and notification sent	Passed
TC09	Unauthorized Access	Access admin route without token	Access denied with "Unauthorized" message	Passed
TC10	Data Retrieval	Fetch donations from backend via API	Returns correct and updated donation data	Passed

6.3 Results and discussions:

- The system testing of **Fund4Life** yielded positive results. All major functionalities performed as expected under various input conditions.
- **Functional testing** confirmed that all features — from registration and login to fund management — were working correctly.
- **Integration testing** verified that communication between the **Flask backend** and **React frontend** using **Axios APIs** was seamless.
- **Validation testing** ensured that incorrect or incomplete inputs (like invalid emails or missing fields) triggered appropriate error messages.
- **Security testing** confirmed that password encryption, JWT authentication, and OTP verification prevented unauthorized access.
- No critical bugs were found after the final testing iteration. Minor UI alignment issues were corrected during frontend validation.

6.4 Performance Evaluation:

Performance testing was conducted to evaluate the speed, stability, and scalability of the Fund4Life system under varying conditions. The following aspects were analyzed:

- **Response** Time:
Average API response time remained under 1 second for most requests, ensuring smooth user interaction.
- **Load** Handling:
The system was tested with 50–100 concurrent requests, and the backend handled them efficiently without database slowdowns.
- **Database** Efficiency:
Queries executed through SQLAlchemy ORM were optimized to reduce response time and prevent redundancy.
- **Security** and Reliability:
OTP-based verification, JWT authentication, and password hashing ensured a high level of system security.
- **Cross-Platform** Performance:
The application was tested across multiple browsers (Chrome, Edge, Firefox) and devices (laptop, mobile), and all performed consistently.

Overall, the system met the required performance benchmarks for a medium-scale community platform.

6.5 Summary:

The system testing phase of the Fund4Life project played a vital role in ensuring the reliability, stability, and functionality of the entire platform. Through detailed functional, integration, and performance testing, every module of the system was thoroughly verified to confirm that it operates according to the defined requirements and objectives.

The testing covered all the major modules of Fund4Life, including user registration, donor login, OTP verification, donation processing, emergency request submission, NGO registration, admin approval, and fund allocation. Each of these components was examined under multiple conditions to validate both correct and erroneous user inputs.

The functional testing confirmed that all core features performed as intended. Users could successfully register, log in, make donations, and request emergency aid. The admin panel effectively handled NGO approvals, case verifications, and fund management without system crashes or inconsistencies.

During integration testing, the communication between the React frontend and Flask backend via Axios API calls was tested thoroughly. Data flow between the client interface and the server remained accurate and consistent. The system properly synchronized updates in real time, such as displaying approved requests or reflecting donation totals instantly.

CHAPTER 7

CONCLUSION & FUTURE ENHANCEMENTS

Conclusion:

The Fund4Life project successfully demonstrates how technology can be leveraged to create a transparent, community-driven donation and aid management platform. The system effectively bridges the gap between donors, recipients, NGOs, and administrators by providing a unified platform for secure donations, real-time case tracking, and verified fund distribution.

By combining a React-based frontend and a Flask-powered backend, the application ensures a smooth user experience with efficient data handling. The modular design supports easy maintenance and future scalability, while database management through MySQL ensures secure and consistent data storage.

The project fulfills its main objectives — enabling individuals to donate effortlessly, allowing emergency patients to request financial help, and empowering admins to verify and allocate funds transparently. The addition of OTP-based login and admin-controlled approvals enhances system security and authenticity.

Overall, Fund4Life represents a complete and functional prototype that promotes trust, accessibility, and social impact within communities. It demonstrates how digital solutions can transform traditional donation processes into efficient and accountable systems for real-world use

Future Enhancements

Although the current version of Fund4Life performs effectively, several enhancements can be implemented to improve its usability, scalability, and intelligence:

- **AI-Based Fund Recommendation:**
Implementing AI and data analytics can automatically suggest optimal fund allocation strategies based on recipient needs, donor history, and urgency of cases.
- **Automated Document Verification:**
Using OCR (Optical Character Recognition) and AI-powered verification to validate uploaded medical and identity documents can reduce manual admin workload.
- **Mobile Application Integration:**
Building an Android/iOS version of Fund4Life will make the platform more accessible to users in rural and remote areas.
- **Blockchain for Transparency:**
Integrating blockchain technology could ensure complete transparency in transactions and prevent any misuse of donated funds.
- **Multi-language and Accessibility Support:**
Adding regional language options and accessibility features (voice input, screen reader compatibility) to reach a larger audience.
- **Chatbot and Support System:**
Implementing an AI chatbot to assist users in donation queries, case updates, or registration issues can improve user engagement.
- **Analytics Dashboard for Admins:**
A more advanced admin dashboard with visual data analytics (graphs, trends, donor patterns) for better decision-making and impact tracking.

With these enhancements, **Fund4Life** can evolve into a **next-generation intelligent donation ecosystem**, combining the best of social welfare and smart technology for maximum real-world impact.

CHAPTER 8

REFERENCES

➤ **MedDonor : Online Medicine Donation Platform.**

MedDonor – An Online Medicine Donation Platform.

JETIR Journal, Volume 10, Issue 4 (2023). Retrieved from

<https://www.jetir.org/papers/JETIR2304B89.pdf>

- ◆ *Limitation:* Focused only on unused medicine exchange without user verification or financial aid.

- ◆ *Fund4Life Improvement:* Adds verified emergency medical donation system with document validation and donor-recipient transparency.

➤ **The Good Way – General Donation Application.**

A Study and Implementation of Donation Application: The Good Way.

International Journal of Scientific Development and Research (IJSDR) (2023). Retrieved from

https://papers.ssrn.com/sol3/Delivery.cfm/SSRN_ID4440085_code1214967.pdf

- ◆ *Limitation:* Generic donation platform without medical categorization or fund tracking.

- ◆ *Fund4Life Improvement:* Introduces category-specific donations (emergency medical, NGO aid) with admin approval and transaction history.

➤ **Online Medicine Donation System for NGOs.**

An Online Web Portal for Donating Unused Medicine to NGOs.

ResearchGate Publication (2022). Retrieved from

<https://www.researchgate.net/publication/361389050>

- ◆ *Limitation:* Manual NGO verification and no donor feedback mechanism.

- ◆ *Fund4Life Improvement:* Implements automated NGO approval, volunteer verification, and AI-based document analysis for authenticity.

➤ **Ketto – Indian Medical Crowdfunding Platform.**

Ketto and the Science of Giving: A Data-Driven Investigation of Crowdfunding for India.

arXiv Preprint (2025). Retrieved from

<https://arxiv.org/abs/2509.12616>

- ◆ *Limitation:* Commercial model with high platform fees and limited transparency for donors.

- ◆ *Fund4Life Improvement:* Fully community-driven, zero-fee model ensuring 100% fund utilization with real-time impact visibility.

➤ **Blockchain-Based Crowdfunding Platform.**

A Blockchain-based Platform for Ensuring Provenance and Transparency in Crowdfunding.
Journal of Blockchain Research (2024). Retrieved from

<https://www.sciencedirect.com/science/article/pii/S2096720925000053>

- ◆ *Limitation:* Focuses only on blockchain verification without user-friendly interfaces.
- ◆ *Fund4Life Improvement:* Combines secure backend verification with intuitive React-based UI, real-time tracking, and admin dashboards.

➤ **Explaining Donation Behavior in Medical Crowdfunding.**

Explaining Donation Behavior in Medical Crowdfunding in Social Media.

University of Twente Research Paper (2021). Retrieved from

https://research.utwente.nl/files/267019524/Huang_2021_Explaining_donation_behavior_in_medical_crowdfunding.pdf

- ◆ *Limitation:* Focused on behavioral analysis without practical system integration.
- ◆ *Fund4Life Improvement:* Integrates behavioral insights into recommendation logic — suggesting verified cases to donors for faster fund allocation